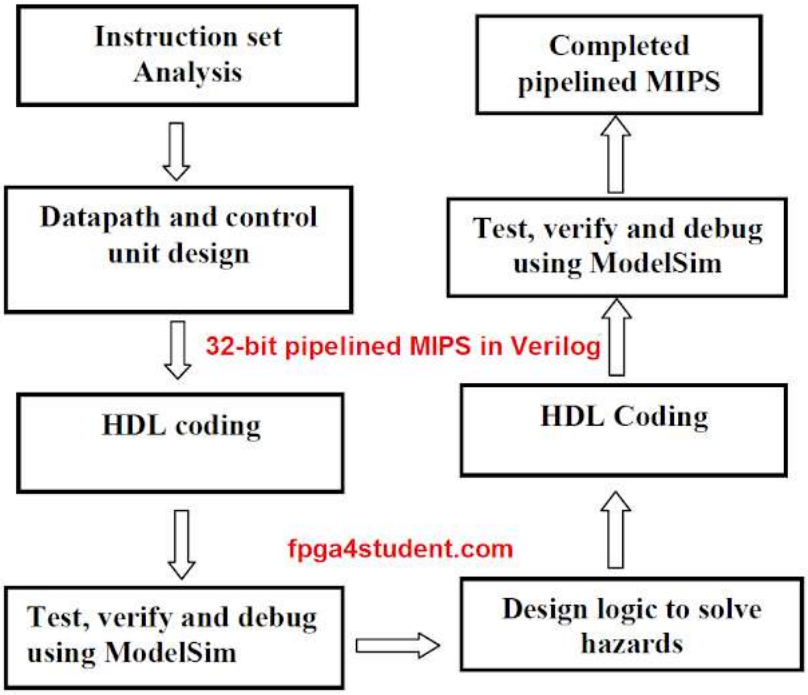
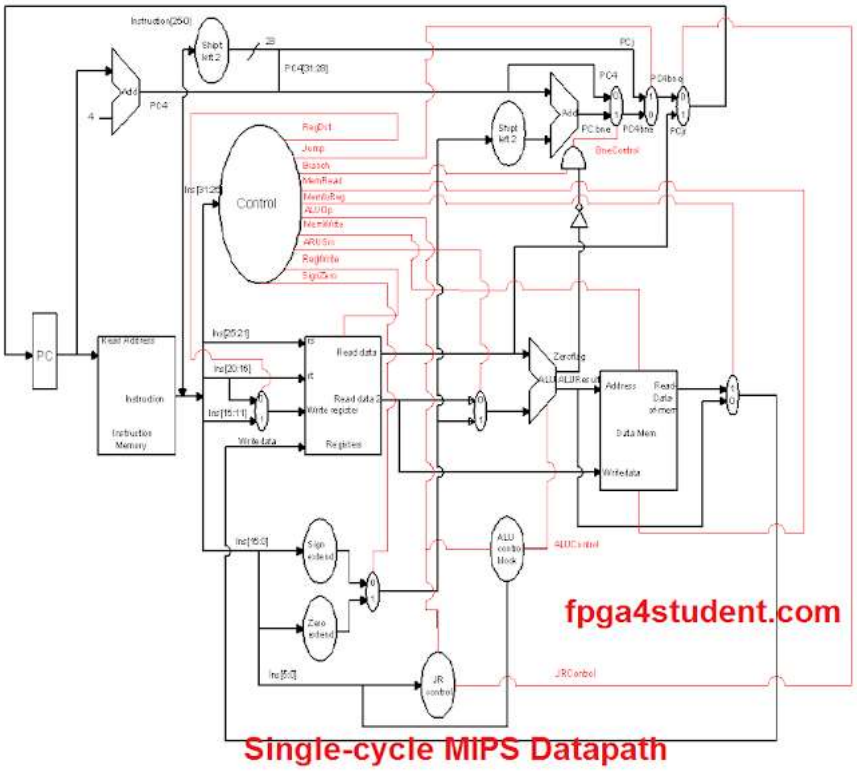
Pipelined MIPS Processor in Verilog (Part-1)

Last time, I posted a Verilog code for a 16-bit single-cycle MIPS Processor and there were several requests for a Verilog code of a 32-bit 5-stage pipelined MIPS Processor. The first problem with the single-cycle MIPS is wasteful of the area which only each functional unit is used once per clock cycle. Another serious drawback is that the clock cycle is determined by the longest possible path in the Processor. Thus, the pipelined MIPS came out to solve those problems by exploiting most functional unit in one clock cycle and improving the performance by increasing the instruction throughput. However, the pipelined MIPS also faces challenges such as control and data hazards.

Today, a 32-bit 5-stage pipelined MIPS Processor will be designed and implemented in Verilog.
Verilog code for special modules such as Forwarding Unit, Flush Control Unit and Stall Control unit for solving hazards will be also provided. The Verilog code for 32-bit pipelined MIPS Processor is mostly done by using structural modeling.
This project is quite long so I will divide it into 3 parts (Part 1, Part 2, and Part 3).



The design flow for the 32-bit pipelined MIPS flow is shown in the above figure. However, at first, the instruction set of the MIPS Processor is as follows:
1. ADD rd, rs, rt: Reg[rd] = Reg[rs] + Reg[rt].
2. BNE rs, rt, imm16: if (Reg[rs] != Reg[rt]) PC = PC + 4 + Sign_ext(Imm16)<<2 else PC = PC + 4.
3. J target: PC = { PC[31:28], target, 00 }.
4. JR rs: PC = Reg[rs].
5. LW rt, imm16(rs): Reg[rt] = Mem[Reg[rs] + Sign_ext(Imm16)].
6. SLT rd, rs, rt: If (Reg[rs] < Reg[rt]) Reg[rd] = 00000001 else Reg[rd] = 00000000.
7. SUB rd, rs, rt: Reg[rd] = Reg[rs] – Reg[rt].
8. SW rt, imm16(rs): Mem[Reg[rs] + Sign_ext(Imm16)] = Reg[rt].
9. XORI rt, rs, imm16: Reg[rt] = Reg[rs] XOR Zero_ext(Imm16).
From the instruction set architecture, the single-cycle datapath with the control unit of the MIPS Processor is obtained as shown below.



Now, presenting the Verilog code for the single-cycle MIPS Processor datapath first.

Verilog code for instruction memory:

```
/* Instruction memory module.  Change the $readmemb line to have the name of the p
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Instruction memory module
`timescale 1 ps / 100 fs
module InstructionMem(instruction, address);

input [31:0] address;
output [31:0] instruction;
reg [31:0]instrmem[1023:0];
reg [31:0] temp;

buf #1000 buf0(instruction[0],temp[0]),
   buf1(instruction[1],temp[1]),
   buf2(instruction[2],temp[2]),
   buf3(instruction[3],temp[3]),
   buf4(instruction[4],temp[4]),
   buf5(instruction[5],temp[5]),
   buf6(instruction[6],temp[6]),
   buf7(instruction[7],temp[7]),
   buf8(instruction[8],temp[8]),
   buf9(instruction[9],temp[9]),
   buf10(instruction[10],temp[10]),
   buf11(instruction[11],temp[11]),
   buf12(instruction[12],temp[12]),
   buf13(instruction[13],temp[13]),
   buf14(instruction[14],temp[14]),
   buf15(instruction[15],temp[15]),
   buf16(instruction[16],temp[16]),
   buf17(instruction[17],temp[17]),
   buf18(instruction[18],temp[18]),
   buf19(instruction[19],temp[19]),
   buf20(instruction[20],temp[20]),
   buf21(instruction[21],temp[21]),
```

```
   buf22(instruction[22],temp[22]),
   buf23(instruction[23],temp[23]),
   buf24(instruction[24],temp[24]),
   buf25(instruction[25],temp[25]),
   buf26(instruction[26],temp[26]),
   buf27(instruction[27],temp[27]),
   buf28(instruction[28],temp[28]),
   buf29(instruction[29],temp[29]),
   buf30(instruction[30],temp[30]),
   buf31(instruction[31],temp[31]);

always @(address)
begin
 temp=instrmem[address/4];
end

initial
begin
$readmemb("instr.txt", instrmem);
end

endmodule


module instrmemstimulous();

reg [31:0] addr;
wire [31:0] instr;

InstructionMem instructionmemory(instr, addr);

initial
begin
$monitor("Mem Address=%h instruction=%b",addr,instr);
addr=32'd0;
#10000 addr=32'd4;
#10000 addr=32'd8;
#10000 addr=32'd12;
#10000 addr=32'd16;
#10000 addr=32'd20;
#10000 addr=32'd24;
#10000 addr=32'd28;
#10000;
$finish;
end

endmodule
```

Instructions that you want to load into the instruction memory need to save in the "*instr.txt*" in the binary format. If you want to save it hexadecimal format, replace the $readmemb instruction by $readmemh instruction in the Verilog code of the instruction memory. Below is an example of the "*instr.txt*" file:

```
00111000000100000000000000000011
00111000000100010000000000000100
00001000000000000000000000000101
00111000000100000000000000000001
00111000000100010000000000000001
00000010001100001001000000100010
00010110000100011111111111111100
00000010000100011001100000100000
10101110010100110000000000010000
10001110010101000000000000010000
```

```
00000010000101001010100000101010
10001110010100110000000000010000
00111010010100110000000000000001
00111010101101010000000000000001
00000010101000000000000000001000
```

Verilog code for 32-bit Adder:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Verilog code for 32-bit adder
module Add(S,A,B);
output [31:0] S;
input [31:0] A,B;
wire [31:0] C;
 adder1bit adder1bit0(S[0],C[0],A[0],B[0],1'b0);
 adder1bit adder1bit1(S[1],C[1],A[1],B[1],C[0]);
 adder1bit adder1bit2(S[2],C[2],A[2],B[2],C[1]);
 adder1bit adder1bit3(S[3],C[3],A[3],B[3],C[2]);
 adder1bit adder1bit4(S[4],C[4],A[4],B[4],C[3]);
 adder1bit adder1bit5(S[5],C[5],A[5],B[5],C[4]);
 adder1bit adder1bit6(S[6],C[6],A[6],B[6],C[5]);
 adder1bit adder1bit7(S[7],C[7],A[7],B[7],C[6]);
 adder1bit adder1bit8(S[8],C[8],A[8],B[8],C[7]);
 adder1bit adder1bit9(S[9],C[9],A[9],B[9],C[8]);
 adder1bit adder1bit10(S[10],C[10],A[10],B[10],C[9]);
 adder1bit adder1bit11(S[11],C[11],A[11],B[11],C[10]);
 adder1bit adder1bit12(S[12],C[12],A[12],B[12],C[11]);
 adder1bit adder1bit13(S[13],C[13],A[13],B[13],C[12]);
 adder1bit adder1bit14(S[14],C[14],A[14],B[14],C[13]);
 adder1bit adder1bit15(S[15],C[15],A[15],B[15],C[14]);
 adder1bit adder1bit16(S[16],C[16],A[16],B[16],C[15]);
 adder1bit adder1bit17(S[17],C[17],A[17],B[17],C[16]);
 adder1bit adder1bit18(S[18],C[18],A[18],B[18],C[17]);
 adder1bit adder1bit19(S[19],C[19],A[19],B[19],C[18]);
 adder1bit adder1bit20(S[20],C[20],A[20],B[20],C[19]);
 adder1bit adder1bit21(S[21],C[21],A[21],B[21],C[20]);
 adder1bit adder1bit22(S[22],C[22],A[22],B[22],C[21]);
 adder1bit adder1bit23(S[23],C[23],A[23],B[23],C[22]);
 adder1bit adder1bit24(S[24],C[24],A[24],B[24],C[23]);
 adder1bit adder1bit25(S[25],C[25],A[25],B[25],C[24]);
 adder1bit adder1bit26(S[26],C[26],A[26],B[26],C[25]);
 adder1bit adder1bit27(S[27],C[27],A[27],B[27],C[26]);
 adder1bit adder1bit28(S[28],C[28],A[28],B[28],C[27]);
 adder1bit adder1bit29(S[29],C[29],A[29],B[29],C[28]);
 adder1bit adder1bit30(S[30],C[30],A[30],B[30],C[29]);
 adder1bit adder1bit31(S[31],C[31],A[31],B[31],C[30]);

endmodule

//-------------------------------------------------------------------------------

`timescale 1 ps / 100 fs
module adder1bit(sum,cout,a,b,cin);
input   a,b,cin;
output  cout,sum;
// sum = a xor b xor cin
xor #(50) (sum,a,b,cin);
// carry out = a.b + cin.(a+b)
and #(50) and1(c1,a,b);
or #(50) or1(c2,a,b);
and #(50) and2(c3,c2,cin);
```

```
or #(50) or2(cout,c1,c3);
endmodule
```

Verilog code for register file:

```
`timescale 1 ps / 100 fs
// fpga4student.com: FPGA projects, Verilog Projects, VHDL projects
// Verilog project: 32-bit 5-stage Pipelined MIPS Processor in Verilog
// Register file
module regfile(
ReadData1,
ReadData2,
WriteData,
ReadRegister1,
ReadRegister2,
WriteRegister,
RegWrite,
reset,
clk);

input [4:0]ReadRegister1,ReadRegister2,WriteRegister;
input [31:0] WriteData;
input RegWrite,reset, clk;
output [31:0] ReadData1, ReadData2;
wire [31:0] WriteEn;
wire [31:0] RegArray [0:31];
integer i;
     //----Decoder Block
 decoder Decoder1( WriteEn,RegWrite,WriteRegister);
 register reg0 (RegArray[0],32'b0,1'b1,1'b0, clk);
 register reg1 (RegArray[1],WriteData,WriteEn[1],reset,clk);
 register reg2 (RegArray[2],WriteData,WriteEn[2],reset,clk);
 register reg3 (RegArray[3],WriteData,WriteEn[3],reset,clk);
 register reg4 (RegArray[4],WriteData,WriteEn[4],reset,clk);
 register reg5 (RegArray[5],WriteData,WriteEn[5],reset,clk);
 register reg6 (RegArray[6],WriteData,WriteEn[6],reset,clk);
 register reg7 (RegArray[7],WriteData,WriteEn[7],reset,clk);
 register reg8 (RegArray[8],WriteData,WriteEn[8],reset,clk);
 register reg9 (RegArray[9],WriteData,WriteEn[9],reset,clk);
 register reg10 (RegArray[10],WriteData,WriteEn[10],reset,clk);
 register reg11 (RegArray[11],WriteData,WriteEn[11],reset,clk);
 register reg12 (RegArray[12],WriteData,WriteEn[12],reset,clk);
 register reg13 (RegArray[13],WriteData,WriteEn[13],reset,clk);
 register reg14 (RegArray[14],WriteData,WriteEn[14],reset,clk);
 register reg15 (RegArray[15],WriteData,WriteEn[15],reset,clk);
 register reg16 (RegArray[16],WriteData,WriteEn[16],reset,clk);
 register reg17 (RegArray[17],WriteData,WriteEn[17],reset,clk);
 register reg18 (RegArray[18],WriteData,WriteEn[18],reset,clk);
 register reg19 (RegArray[19],WriteData,WriteEn[19],reset,clk);
 register reg20 (RegArray[20],WriteData,WriteEn[20],reset,clk);
 register reg21 (RegArray[21],WriteData,WriteEn[21],reset,clk);
 register reg22 (RegArray[22],WriteData,WriteEn[22],reset,clk);
 register reg23 (RegArray[23],WriteData,WriteEn[23],reset,clk);
 register reg24 (RegArray[24],WriteData,WriteEn[24],reset,clk);
 register reg25 (RegArray[25],WriteData,WriteEn[25],reset,clk);
 register reg26 (RegArray[26],WriteData,WriteEn[26],reset,clk);
 register reg27 (RegArray[27],WriteData,WriteEn[27],reset,clk);
 register reg28 (RegArray[28],WriteData,WriteEn[28],reset,clk);
 register reg29 (RegArray[29],WriteData,WriteEn[29],reset,clk);
 register reg30 (RegArray[30],WriteData,WriteEn[30],reset,clk);
 register reg31 (RegArray[31],WriteData,WriteEn[31],reset,clk);
 //----32x32to32 Multiplexor1 Block----
    mux32x32to32 Mux1(ReadData1,RegArray[0], RegArray[1],RegArray[2], RegArray[3],
 RegArray[8],RegArray[9],RegArray[10],RegArray[11],RegArray[12],RegArray[13],RegAr
```

```
 RegArray[18], RegArray[19],RegArray[20],RegArray[21],RegArray[22],RegArray[23],Re
 RegArray[27], RegArray[28], RegArray[29],RegArray[30],RegArray[31], ReadRegister1
      );

 //----32x32to32 Multiplexor2 Block----
 mux32x32to32 Mux2(ReadData2,RegArray[0], RegArray[1],RegArray[2], RegArray[3],Reg
 RegArray[8],RegArray[9],RegArray[10],RegArray[11],RegArray[12],RegArray[13],RegAr
 RegArray[18], RegArray[19],RegArray[20],RegArray[21],RegArray[22],RegArray[23],Re
 RegArray[27], RegArray[28], RegArray[29],RegArray[30],RegArray[31], ReadRegister2
       );
endmodule

//------------DFF-------------------
module D_FF (q, d, reset, clk);
output q;
input d, reset, clk;
reg q; // Indicate that q is stateholding

always @(posedge clk or posedge reset)
if (reset)
q = 0; // On reset, set to 0
else
q = d; // Otherwise out = d
endmodule
// 1 bit register
module RegBit(BitOut, BitData, WriteEn,reset, clk);
output BitOut; // 1 bit of register
input BitData, WriteEn;
input reset,clk;
wire d,f1, f2; // input of D Flip-Flop
wire reset;
//assign reset=0;
and #(50) U1(f1, BitOut, (~WriteEn));
and #(50) U2(f2, BitData, WriteEn);
or  #(50) U3(d, f1, f2);
D_FF DFF0(BitOut, d, reset, clk);
endmodule

//32 bit register
module register(RegOut,RegIn,WriteEn,reset,clk);
output [31:0] RegOut;
input [31:0] RegIn;
input WriteEn,reset, clk;
RegBit bit31(RegOut[31],RegIn[31],WriteEn,reset,clk);
RegBit bit30(RegOut[30],RegIn[30],WriteEn,reset,clk);
RegBit bit29(RegOut[29],RegIn[29],WriteEn,reset,clk);
RegBit bit28(RegOut[28],RegIn[28],WriteEn,reset,clk);
RegBit bit27(RegOut[27],RegIn[27],WriteEn,reset,clk);
RegBit bit26(RegOut[26],RegIn[26],WriteEn,reset,clk);
RegBit bit25(RegOut[25],RegIn[25],WriteEn,reset,clk);
RegBit bit24(RegOut[24],RegIn[24],WriteEn,reset,clk);
RegBit bit23(RegOut[23],RegIn[23],WriteEn,reset,clk);
RegBit bit22(RegOut[22],RegIn[22],WriteEn,reset,clk);
RegBit bit21(RegOut[21],RegIn[21],WriteEn,reset,clk);
RegBit bit20(RegOut[20],RegIn[20],WriteEn,reset,clk);
RegBit bit19(RegOut[19],RegIn[19],WriteEn,reset,clk);
RegBit bit18(RegOut[18],RegIn[18],WriteEn,reset,clk);
RegBit bit17(RegOut[17],RegIn[17],WriteEn,reset,clk);
RegBit bit16(RegOut[16],RegIn[16],WriteEn,reset,clk);
RegBit bit15(RegOut[15],RegIn[15],WriteEn,reset,clk);
RegBit bit14(RegOut[14],RegIn[14],WriteEn,reset,clk);
RegBit bit13(RegOut[13],RegIn[13],WriteEn,reset,clk);
RegBit bit12(RegOut[12],RegIn[12],WriteEn,reset,clk);
```

```
RegBit bit11(RegOut[11],RegIn[11],WriteEn,reset,clk);
RegBit bit10(RegOut[10],RegIn[10],WriteEn,reset,clk);
RegBit bit9 (RegOut[9], RegIn[9], WriteEn,reset,clk);
RegBit bit8 (RegOut[8], RegIn[8], WriteEn,reset,clk);
RegBit bit7 (RegOut[7], RegIn[7], WriteEn,reset,clk);
RegBit bit6 (RegOut[6], RegIn[6], WriteEn,reset,clk);
RegBit bit5 (RegOut[5], RegIn[5], WriteEn,reset,clk);
RegBit bit4 (RegOut[4], RegIn[4], WriteEn,reset,clk);
RegBit bit3 (RegOut[3], RegIn[3], WriteEn,reset,clk);
RegBit bit2 (RegOut[2], RegIn[2], WriteEn,reset,clk);
RegBit bit1 (RegOut[1], RegIn[1], WriteEn,reset,clk);
RegBit bit0 (RegOut[0], RegIn[0], WriteEn,reset,clk);

endmodule

// Decoder
module decoder(WriteEn,RegWrite, WriteRegister);
input RegWrite;
input [4:0] WriteRegister;
output [31:0] WriteEn;
wire [31:0] OE; // Output Enable
dec5to32 dec(OE,WriteRegister);
assign WriteEn[0]=0;
 and  #(50) gate1(WriteEn[1],OE[1],RegWrite);
 and  #(50) gate2(WriteEn[2],OE[2],RegWrite);
 and  #(50) gate3(WriteEn[3],OE[3],RegWrite);
 and  #(50) gate4(WriteEn[4],OE[4],RegWrite);
 and  #(50) gate5(WriteEn[5],OE[5],RegWrite);
 and  #(50) gate6(WriteEn[6],OE[6],RegWrite);
 and  #(50) gate7(WriteEn[7],OE[7],RegWrite);
 and  #(50) gate8(WriteEn[8],OE[8],RegWrite);
 and  #(50) gate9(WriteEn[9],OE[9],RegWrite);
 and  #(50) gate10(WriteEn[10],OE[10],RegWrite);
 and  #(50) gate11(WriteEn[11],OE[11],RegWrite);
 and  #(50) gate12(WriteEn[12],OE[12],RegWrite);
 and  #(50) gate13(WriteEn[13],OE[13],RegWrite);
 and  #(50) gate14(WriteEn[14],OE[14],RegWrite);
 and  #(50) gate15(WriteEn[15],OE[15],RegWrite);
 and  #(50) gate16(WriteEn[16],OE[16],RegWrite);
 and  #(50) gate17(WriteEn[17],OE[17],RegWrite);
 and  #(50) gate18(WriteEn[18],OE[18],RegWrite);
 and  #(50) gate19(WriteEn[19],OE[19],RegWrite);
 and  #(50) gate20(WriteEn[20],OE[20],RegWrite);
 and  #(50) gate21(WriteEn[21],OE[21],RegWrite);
 and  #(50) gate22(WriteEn[22],OE[22],RegWrite);
 and  #(50) gate23(WriteEn[23],OE[23],RegWrite);
 and  #(50) gate24(WriteEn[24],OE[24],RegWrite);
 and  #(50) gate25(WriteEn[25],OE[25],RegWrite);
 and  #(50) gate26(WriteEn[26],OE[26],RegWrite);
 and  #(50) gate27(WriteEn[27],OE[27],RegWrite);
 and  #(50) gate28(WriteEn[28],OE[28],RegWrite);
 and  #(50) gate29(WriteEn[29],OE[29],RegWrite);
 and  #(50) gate30(WriteEn[30],OE[30],RegWrite);
 and  #(50) gate31(WriteEn[31],OE[31],RegWrite);
endmodule
module andmore(g,a,b,c,d,e);
  output g;
  input a,b,c,d,e;
  and #(50) and1(f1,a,b,c,d),
            and2(g,f1,e);
endmodule
module dec5to32(Out,Adr);
input [4:0] Adr; // Adr=Address of register
```

```
output [31:0] Out;
not #(50) Inv4(Nota, Adr[4]);
not #(50) Inv3(Notb, Adr[3]);
not #(50) Inv2(Notc, Adr[2]);
not #(50) Inv1(Notd, Adr[1]);
not #(50) Inv0(Note, Adr[0]);

andmore a0(Out[0],  Nota,Notb,Notc,Notd,Note); // 00000
andmore a1(Out[1],  Nota,Notb,Notc,Notd,Adr[0]); // 00001
andmore a2(Out[2],  Nota,Notb,Notc,Adr[1],Note); //00010
andmore a3(Out[3],  Nota,Notb,Notc,Adr[1],Adr[0]);
andmore a4(Out[4],  Nota,Notb,Adr[2],Notd,Note);
andmore a5(Out[5],  Nota,Notb,Adr[2],Notd,Adr[0]);
andmore a6(Out[6],  Nota,Notb,Adr[2],Adr[1],Note);
andmore a7(Out[7],  Nota,Notb,Adr[2],Adr[1],Adr[0]);
andmore a8(Out[8],    Nota,Adr[3],Notc,Notd,Note);
andmore a9(Out[9],    Nota,Adr[3],Notc,Notd,Adr[0]);
andmore a10(Out[10],  Nota,Adr[3],Notc,Adr[1],Note);
andmore a11(Out[11],  Nota,Adr[3],Notc,Adr[1],Adr[0]);
andmore a12(Out[12],  Nota,Adr[3],Adr[2],Notd,Note);
andmore a13(Out[13],  Nota,Adr[3],Adr[2],Notd,Adr[0]);
andmore a14(Out[14],  Nota,Adr[3],Adr[2],Adr[1],Note);
andmore a15(Out[15],  Nota,Adr[3],Adr[2],Adr[1],Adr[0]);
andmore a16(Out[16],  Adr[4],Notb,Notc,Notd,Note);
andmore a17(Out[17],  Adr[4],Notb,Notc,Notd,Adr[0]);
andmore a18(Out[18],  Adr[4],Notb,Notc,Adr[1],Note);
andmore a19(Out[19],  Adr[4],Notb,Notc,Adr[1],Adr[0]);
andmore a20(Out[20],  Adr[4],Notb,Adr[2],Notd,Note);
andmore a21(Out[21],  Adr[4],Notb,Adr[2],Notd,Adr[0]);
andmore a22(Out[22],  Adr[4],Notb,Adr[2],Adr[1],Note);
andmore a23(Out[23],  Adr[4],Notb,Adr[2],Adr[1],Adr[0]);
andmore a24(Out[24],  Adr[4],Adr[3],Notc,Notd,Note);
andmore a25(Out[25],  Adr[4],Adr[3],Notc,Notd,Adr[0]);
andmore a26(Out[26],  Adr[4],Adr[3],Notc,Adr[1],Note);
andmore a27(Out[27],  Adr[4],Adr[3],Notc,Adr[1],Adr[0]);
andmore a28(Out[28],  Adr[4],Adr[3],Adr[2],Notd,Note);
andmore a29(Out[29],  Adr[4],Adr[3],Adr[2],Notd,Adr[0]);
andmore a30(Out[30],  Adr[4],Adr[3],Adr[2],Adr[1],Note);
andmore a31(Out[31],  Adr[4],Adr[3],Adr[2],Adr[1],Adr[0]); // 11111
endmodule

//------------module multiplexor 32 to 1----------------
module mux32to1(Out, In , Select);
output Out;
input [31:0] In;
input [4:0] Select;
wire [31:0] OE,f; // OE = Output Enable
dec5to32 dec1(OE,Select);

 and  #(50) g_0(f[0],OE[0],In[0]);
 and  #(50) g_1(f[1],OE[1],In[1]);
 and  #(50) g_2(f[2],OE[2],In[2]);
 and  #(50) g_3(f[3],OE[3],In[3]);
 and  #(50) g_4(f[4],OE[4],In[4]);
 and  #(50) g_5(f[5],OE[5],In[5]);
 and  #(50) g_6(f[6],OE[6],In[6]);
 and  #(50) g_7(f[7],OE[7],In[7]);
 and  #(50) g_8(f[8],OE[8],In[8]);
 and  #(50) g_9(f[9],OE[9],In[9]);
 and  #(50) g_10(f[10],OE[10],In[10]);
 and  #(50) g_11(f[11],OE[11],In[11]);
 and  #(50) g_12(f[12],OE[12],In[12]);
 and  #(50) g_13(f[13],OE[13],In[13]);
```

```
 and  #(50) g_14(f[14],OE[14],In[14]);
 and  #(50) g_15(f[15],OE[15],In[15]);
 and  #(50) g_16(f[16],OE[16],In[16]);
 and  #(50) g_17(f[17],OE[17],In[17]);
 and  #(50) g_18(f[18],OE[18],In[18]);
 and  #(50) g_19(f[19],OE[19],In[19]);
 and  #(50) g_20(f[20],OE[20],In[20]);
 and  #(50) g_21(f[21],OE[21],In[21]);
 and  #(50) g_22(f[22],OE[22],In[22]);
 and  #(50) g_23(f[23],OE[23],In[23]);
 and  #(50) g_24(f[24],OE[24],In[24]);
 and  #(50) g_25(f[25],OE[25],In[25]);
 and  #(50) g_26(f[26],OE[26],In[26]);
 and  #(50) g_27(f[27],OE[27],In[27]);
 and  #(50) g_28(f[28],OE[28],In[28]);
 and  #(50) g_29(f[29],OE[29],In[29]);
 and  #(50) g_30(f[30],OE[30],In[30]);
 and  #(50) g_31(f[31],OE[31],In[31]);



 or #(50) gate3(g3,f[0],f[1],f[2],f[3]);
 or #(50) gate4(g4,f[4],f[5],f[6],f[7]);
 or #(50) gate5(g5,f[8],f[9],f[10],f[11]);
 or #(50) gate6(g6,f[12],f[13],f[14],f[15]);
 or #(50) gate7(g7,f[16],f[17],f[18],f[19]);
 or #(50) gate8(g8,f[20],f[21],f[22],f[23]);
 or #(50) gate9(g9,f[24],f[25],f[26],f[27]);
 or #(50) gate10(g10,f[28],f[29],f[30],f[31]);
 or #(50) gate11(g11,g3,g4,g5,g6);
 or #(50) gate12(g12,g7,g8,g9,10);
 or #(50) gate(Out,g11,g12);
 endmodule
module mux32x32to32(ReadData,In0, In1,In2,In3,In4,In5,In6,In7,In8,In9,In10,In11,In

input [31:0] In0, In1,In2,In3,In4,In5,In6,In7,In8,In9,In10,In11,In12,In13,In14,In1
input [4:0] ReadRegister;
output [31:0] ReadData;
reg [31:0] ArrayReg [0:31];
integer j;
always @(*)
begin
for (j=0;j<=31;j=j+1)
          ArrayReg[j] = {In31[j], In30[j],In29[j],In28[j],In27[j],In26[j],In25[j],
       In20[j],In19[j],In18[j],In17[j],In16[j],In15[j],In14[j],In13[j],In12[j],In1
       In10[j],In9[j],In8[j],In7[j],In6[j],In5[j],In4[j],In3[j],In2[j],In1[j],In0[

end
mux32to1  mux0(ReadData[0],ArrayReg[0],ReadRegister);
mux32to1  mux1(ReadData[1],ArrayReg[1],ReadRegister);
mux32to1  mux2(ReadData[2],ArrayReg[2],ReadRegister);
mux32to1  mux3(ReadData[3],ArrayReg[3],ReadRegister);
mux32to1  mux4(ReadData[4],ArrayReg[4],ReadRegister);
mux32to1  mux5(ReadData[5],ArrayReg[5],ReadRegister);
mux32to1  mux6(ReadData[6],ArrayReg[6],ReadRegister);
mux32to1  mux7(ReadData[7],ArrayReg[7],ReadRegister);
mux32to1  mux8(ReadData[8],ArrayReg[8],ReadRegister);
mux32to1  mux9(ReadData[9],ArrayReg[9],ReadRegister);
mux32to1  mux10(ReadData[10],ArrayReg[10],ReadRegister);
mux32to1  mux11(ReadData[11],ArrayReg[11],ReadRegister);
mux32to1  mux12(ReadData[12],ArrayReg[12],ReadRegister);
mux32to1  mux13(ReadData[13],ArrayReg[13],ReadRegister);
mux32to1  mux14(ReadData[14],ArrayReg[14],ReadRegister);
```

```
mux32to1  mux15(ReadData[15],ArrayReg[15],ReadRegister);
mux32to1  mux16(ReadData[16],ArrayReg[16],ReadRegister);
mux32to1  mux17(ReadData[17],ArrayReg[17],ReadRegister);
mux32to1  mux18(ReadData[18],ArrayReg[18],ReadRegister);
mux32to1  mux19(ReadData[19],ArrayReg[19],ReadRegister);
mux32to1  mux20(ReadData[20],ArrayReg[20],ReadRegister);
mux32to1  mux21(ReadData[21],ArrayReg[21],ReadRegister);
mux32to1  mux22(ReadData[22],ArrayReg[22],ReadRegister);
mux32to1  mux23(ReadData[23],ArrayReg[23],ReadRegister);
mux32to1  mux24(ReadData[24],ArrayReg[24],ReadRegister);
mux32to1  mux25(ReadData[25],ArrayReg[25],ReadRegister);
mux32to1  mux26(ReadData[26],ArrayReg[26],ReadRegister);
mux32to1  mux27(ReadData[27],ArrayReg[27],ReadRegister);
mux32to1  mux28(ReadData[28],ArrayReg[28],ReadRegister);
mux32to1  mux29(ReadData[29],ArrayReg[29],ReadRegister);
mux32to1  mux30(ReadData[30],ArrayReg[30],ReadRegister);
mux32to1  mux31(ReadData[31],ArrayReg[31],ReadRegister);

endmodule
```

Continue on the ALU design and Verilog code for the ALU. Let's move to the next part.

It is noted that you need to go through all the necessary parts(Part 1, Part 2, and Part 3) to fully understand the process of designing the pipelined MIPS processor, and collect all the required Verilog code to be able to run the pipelined MIPS processor in simulation.

You may like this:

Verilog code for a Microcontroller
A complete 8-bit Microcontroller in VHDL
Single-cycle MIPS Processor in Verilog
Single-cycle MIPS processor in VHDL
Verilog Code for 16-bit RISC Processor
What is FPGA Programming? FPGA vs Software programming
Recommended and affordable Xilinx FPGA boards for students
Recommended and affordable Altera FPGA boards for students

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports
24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)

26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. **How to generate a clock enable signal in Verilog**
34. **Verilog code for PWM Generator**
35. **Verilog coding vs Software Programming**
36. **Verilog code for Sequence Detector using Moore FSM**

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.